

THREAT INTELLIGENCE PLATFORM · SDG 16

# ScamGraph

실시간 사기·피싱 위협 인텔리전스 플랫폼

의심스러운 URL·전화번호·계좌를 넣으면 **설명 가능한 규칙 엔진**으로 위험을 분석하고, 사기 인프라의 연결망을 **그래프·지도·실시간 피드**로 펼쳐 보여주는 디지털 시민 방어 플랫폼.

Next.js

Spring Boot

Neo4j

Meilisearch

deck.gl

WebSocket

Browser Extension

**행사** · 메이커톤 (해커톤 계열)

**주제** · 디지털 시민으로서 안전한 디지털 세상을 만드는 방안 (SDG 16 · 10)

**문서 목적** · 팀원 온보딩 — 이 문서만으로 프로젝트 전체를 이해

**작성일** · 2026-07-11 · **버전** · v0.1

# 목차

---

1. 개요 & 목적
2. 아키텍처
3. 기술 스택
4. 기능 카탈로그
5. 데이터 흐름 & 동작 원리
6. API 레퍼런스
7. 디렉터리 구조
8. 실행 가이드
9. 파트 분담
- .0. 데모 가이드 (킬샷)
- .1. 데모 세이프 원칙

## 01 개요 & 목적

ScamGraph는 사기·피싱 위협을 실시간으로 탐지·시각화하는 웹 플랫폼입니다. 메이커톤 주제인 “안전한 디지털 세상 (SDG 16)”을 정면으로 겨냥합니다.

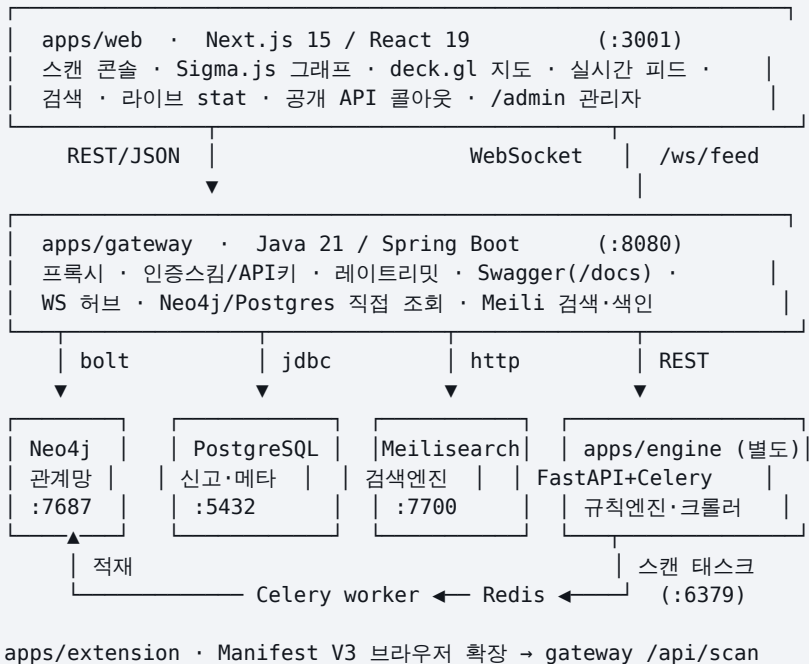
**한 줄 정의** — 시민이 의심스러운 URL·전화·계좌를 검사하면, 규칙 엔진이 위험도와 근거를 즉시 제시하고, 그 대상이 속한 사기 조직 인프라의 관계망을 그래프·지도·실시간 피드로 드러낸다.

### 핵심 차별점

- **설명 가능성** — 블랙박스 AI가 아니라 규칙 기반. 모든 판정에 “왜 위험한지” 근거(rule·weight·detail)가 붙는다. 신뢰가 주어진 만큼 이 자체가 세일즈 포인트.
- **관계망 인사이트** — 개별 대상이 아니라 **사기 인프라의 연결**을 본다. 서로 다른 캠페인이 **같은 IP를 공유**한다는 단서를 시각적으로 드러낸다.
- **플랫폼 규모** — 도구 하나가 아니라 스캔·그래프·피드·지도·검색·관리자·공개 API·브라우저 확장이 물려 돌아가는 생태계.
- **순수 SW** — AI/ML 없이 규칙·공개데이터·그래프로 구현.

## 02 아키텍처

언어·역할이 분리된 **폴리글랏 마이크로서비스** 모노레포. `docker-compose` 한 번으로 8개 서비스가 뜬다.



**설계 원칙 — 실데이터 & 역할 분리.** 게이트웨이는 Neo4j·Postgres를 읽고(그래프·신고·분석), 엔진의 워커는 스캔 결과를 Neo4j에 쓴다. 읽기/쓰기가 분리돼 있어 엔진(담당자 코드)과 게이트웨이가 독립적으로 발전한다.

## 03 기술 스택

| 서비스         | 스택                                                                                                    | 포트        | 역할                            |
|-------------|-------------------------------------------------------------------------------------------------------|-----------|-------------------------------|
| web         | Next.js 15 · React 19 · TypeScript · Sigma.js · graphology · deck.gl · framer-motion · Recharts       | 3001      | 관제 대시보드 · 시각화 · 관리자           |
| gateway     | Java 21 · Spring Boot 3.4 · springdoc(OpenAPI) · Neo4j Java Driver · Spring JDBC · Apache HttpClient5 | 8080      | API 게이트웨이 · 검색 · 실데이터 · WS 허브 |
| engine *    | Python · FastAPI · Celery · httpx · dnspython · python-whois                                          | 8000      | 규칙 스캔엔진 · 크롤링 (담당자 코드)        |
| worker *    | Python · Celery                                                                                       | —         | 비동기 크롤링 · Neo4j 적재            |
| neo4j       | Neo4j 5 Community                                                                                     | 7474/7687 | 관계망 그래프 DB                    |
| postgres    | PostgreSQL 16                                                                                         | 5432      | 신고·스캔·API키 메타                 |
| meilisearch | Meilisearch v1.10                                                                                     | 7700      | 자체 검색엔진                       |
| redis       | Redis 7                                                                                               | 6379      | Celery 브로커 · 캐시               |

\* engine·worker는 Python 담당자 코드로 대체 예정이라 저장소에는 포함되지 않습니다(9장 참조). 로컬 dev/test 코드는 디스크에 존재해 `docker compose` 로 실행됩니다.

## 04 기능 카탈로그

### ① 설명 가능한 규칙 스캔엔진 (순수 SW)

URL/전화/계좌를 입력 → 규칙별 점수 합산 → 등급(**danger/warning**/주의/안전)과 근거 목록 반환. 규칙: 유니코드(호모그래프), 타이포스쿼팅(편집거리), 브랜드 사칭, 피싱 키워드, IP 호스트, 위험 TLD, VoIP 번호 등. `quick_assess()` 는 네트워크 없이 즉답(데모 세이프), 이후 크롤러가 리다이렉트·WHOIS·DNS를 수집.

### ② 관계망 그래프 (Sigma.js)

캠페인·대상·호스트·IP·전화·계좌·신고 노드를 force-directed로 렌더. **공유 IP 허브**가 두 사기 캠페인을 잇는 결정적 단서를 시각적으로 드러낸다(HOSTED\_ON 엽지 강조). hover 포커스·클릭 상세·범례 포함.

### ③ 실시간 신고 피드 (WebSocket)

게이트웨이 `/ws/feed` 허브가 신고·스캔 이벤트를 전 클라이언트에 브로드캐스트. 4초 하트비트로 항상 살아있고, 실제 스캔도 피드에 흘러든다. 프론트는 자동 재연결 + 시드로 데모 세이프.

### ④ GPU 위험 지도 (deck.gl)

여러 출발점에서 타깃(서울)으로 수렴하는 공격 궤적(ArcLayer)과 출발점(ScatterplotLayer)을 GPU로 렌더. 외부 타일 없이 오프라인 동작. 카스퍼스키 어택맵 류의 비주얼.

### ⑤ 자체 검색엔진 (Meilisearch)

기동 시 위험 카탈로그를 Meili 인덱스에 색인 → `/api/search` 로 전문검색. 웹은 250ms 디바운스 인스턴트 검색. Meili 다운 시 카탈로그 부분매칭 폴백.

### ⑥ 관리자 대시보드 (/admin)

Recharts 차트 3종(등급별 도넛 · 유형별 막대 · 위험 추이 에어리어) + 신고 모더레이션 테이블(확인/반려, 낙관적 업데이트). 유형별 집계는 Postgres 실데이터.

### ⑦ 공개 API (키·Swagger·레이트리밋)

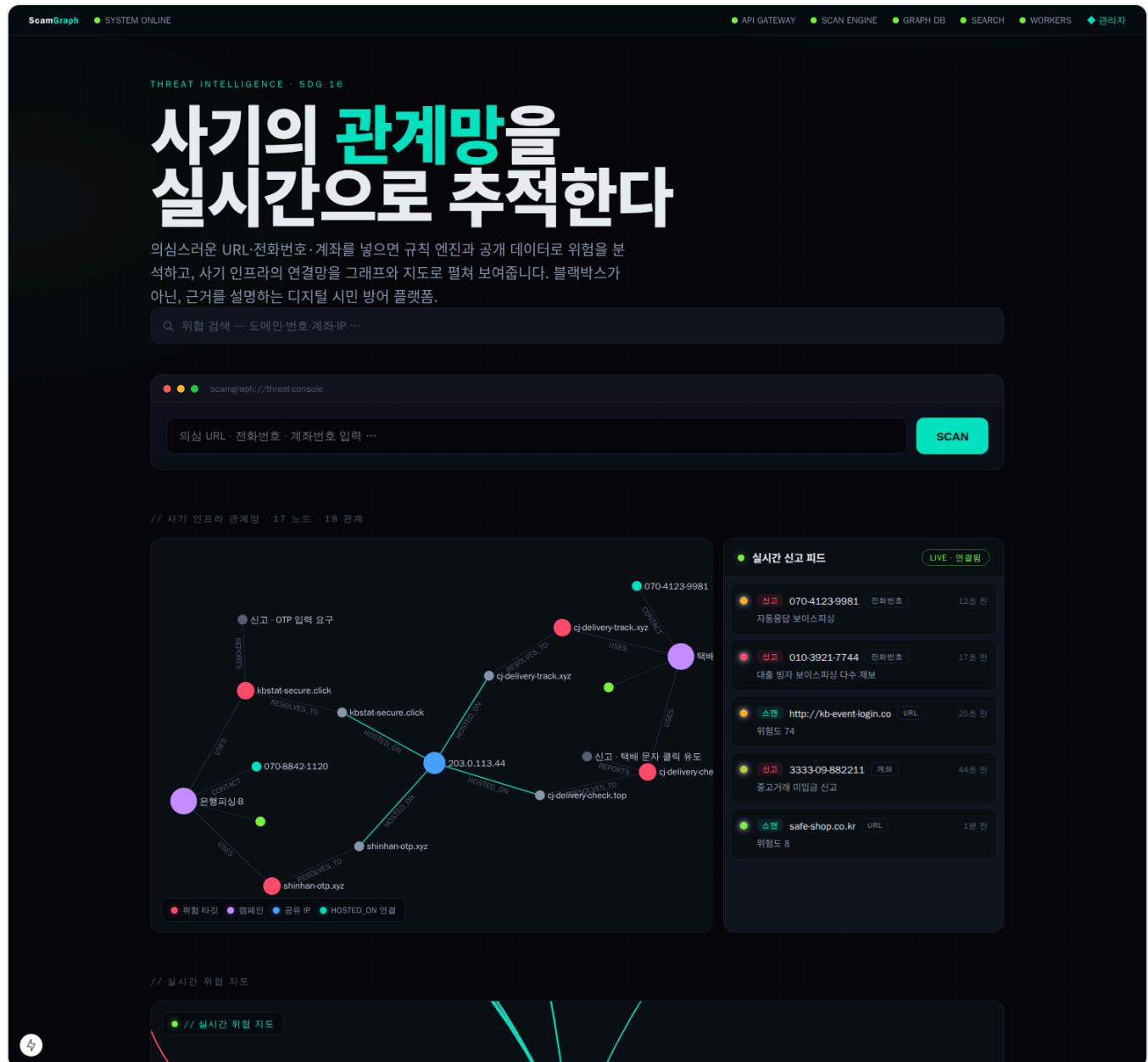
API 키 발급( `/api/keys` ), OpenAPI 문서 `/docs` (Swagger UI, ApiKey 스킴), IP당 토큰버킷 레이트리밋( `X-RateLimit-*` 헤더, 600/분). “다른 개발자가 우리 데이터를 조회”하는 플랫폼 신뢰도.

## ⑧ 브라우저 확장 (Manifest V3)

방문 사이트를 content→background→gateway로 검사 → 위험(danger/warning) 시 경고 배너 주입. 팝업에서 현재 사이트 위험 페이지 확인. XSS 안전(textContent), 게이트웨이 다운 시 무해한 무동작.

## ⑨ 실데이터 연동 (게이트웨이 ↔ Neo4j/Postgres)

게이트웨이가 시드 폴백 대신 **Neo4j·Postgres**를 직접 조회. 스캔하면 워커가 Neo4j에 적재 → 그래프가 **영구 성장**. 비었거나 다운이면 시드로 폴백.



메인 대시보드 — 검색 바 · 스캔 콘솔 · 관계망 그래프(공유 IP 허브) · 실시간 신고 피드(LIVE)

## OPERATOR CONSOLE

## 위협 관제 콘솔

커뮤니티 신고를 검증하고, 등급·유형·추이 분석으로 사기 인프라의 흐름을 한눈에 관제합니다.

총 신고

3

커뮤니티 접수 누적

확인된 위협

3

검증 완료 만장

총 스캔

1,522

엔진 분석 누적

## 위협 분석

등급별 분포



● 위험 ● 경고 ● 주의 ● 안전

유형별 신고



위협 접수 추이



## 신고 모더레이션

| 대상                    | 유형   | 메모           | 상태  | 투표 | 액션    |
|-----------------------|------|--------------|-----|----|-------|
| cj-delivery-check.top | URL  | 택배 미수령 사칭 문자 | 확인됨 | 42 | 확인 반려 |
| kbstat-secure.click   | URL  | KB 보안 인증 사칭  | 확인됨 | 37 | 확인 반려 |
| 070-4123-9981         | 전화번호 | 자동응답 보이스피싱   | 확인됨 | 18 | 확인 반려 |



THREAT INTELLIGENCE · SDG 16

# 사기의 관계망을 실시간으로 추적한다

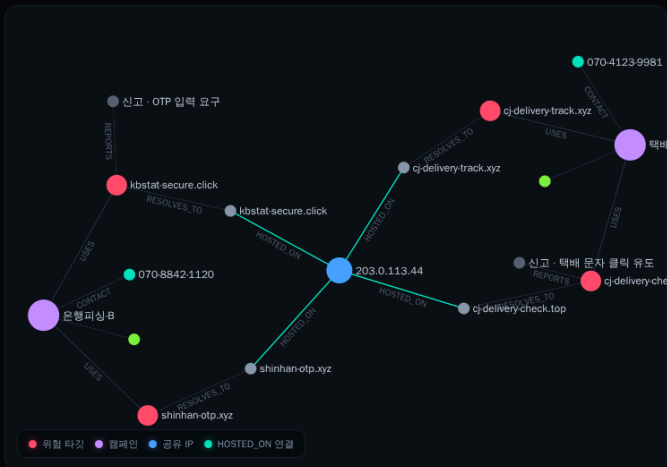
의심스러운 URL·전화번호·계좌를 넣으면 규칙 엔진과 공개 데이터로 위험을 분석하고, 사기 인프라의 연결망을 그래프와 지도로 펼쳐 보여줍니다. 블랙박스가 아닌, 근거를 설명하는 디지털 시인 방어 플랫폼.

scamgraph://threat console

의심 URL · 전화번호 · 계좌번호 입력 ...

SCAN

// 사기 인프라 관계망 · 17 노드 · 18 관계



## 실시간 신고 피드

LIVE · 연결됨

- 신고 010-3921-7744

전화번호

19초 전

대출 빚자 보이스피싱 다수 제보
- 스캠 http://kb-event-login.co

URL

27초 전

위험도 74
- 신고 3333-09-882211

계좌

46초 전

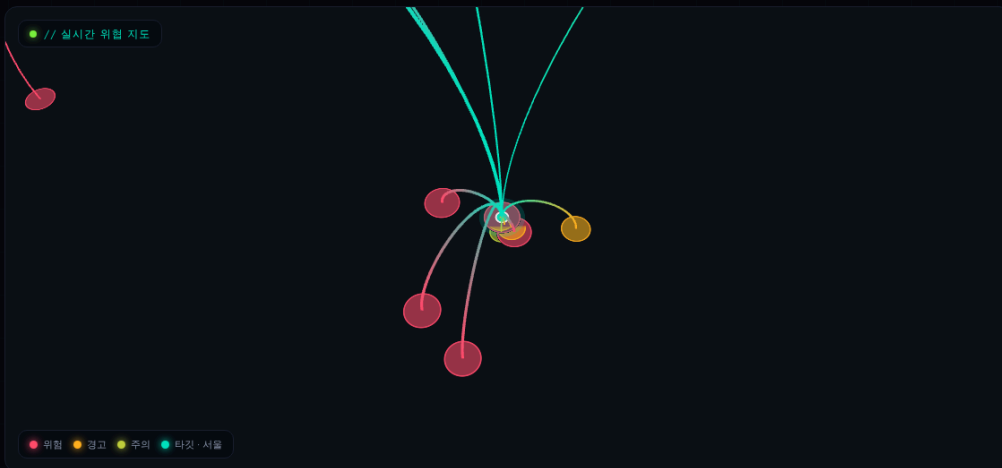
중고거래 미입금 신고
- 스캠 safe-shop.co.kr

URL

1분 전

위험도 8

// 실시간 위험 지도



TRACKED ENTITIES

18,204

▲ 신규 342 / 24h

GRAPH RELATIONS

47,891

도메인 번호 계좌 IP

SCANS TODAY

1,247

▲ 실시간 유입 중

CONFIRMED THREATS

3,516

커뮤니티 검증 완료

// 처리 파이프라인

web

Next.js · React

TS

gateway

Spring Boot · 가상스레드

JAVA 21

engine

FastAPI · 크롤링/규칙

PYTHON

worker

Celery · 비동기 취재

PYTHON

neo4j

관계망 그래프

CYPHER

meilisearch

자체 검색엔진

SEARCH



실시간 신고 피드(WebSocket)와 deck.gl GPU 위협 지도

## 05 데이터 흐름 & 동작 원리

### 스캔 → 그래프 성장 (킬샷의 원리)

1. 웹 스캔 콘솔 입력 → `POST /api/gateway/scan`
2. 게이트웨이 → 엔진 `/scan` 위임 → 규칙 엔진이 위험도+근거 즉시 반환
3. 엔진이 Celery 태스크 트리거 → 워커가 크롤링(리다이렉트/WHOIS/DNS) 후 **Neo4j에 노드 적재**
4. 게이트웨이 `/api/graph` 가 Neo4j를 읽어 새 노드 포함 → 그래프가 자란다
5. 동시에 스캔 이벤트가 `/ws/feed` 로 브로드캐스트 → 실시간 피드에 등장

검증 예: 신규 도메인 스캔 후 Neo4j 노드 17→19, `/api/graph` 에 해당 노드 포함 확인.

### 검색 색인

게이트웨이 기동(ApplicationReadyEvent) → 데몬 스레드가 위협 카탈로그를 Meili `threats` 인덱스에 색인(재시도). Meili primaryKey 제약(영숫자) 때문에 별도 `key` 필드 사용.

### 실시간 피드

`FeedHandler` 가 세션 집합을 관리( `CopyOnWriteArraySet` ) → 4초 하트비트 + 스캔 이벤트를 JSON `TextMessage` 로 전 세션에 전송. 죽은 세션은 개별 try/catch로 건너뛴.

## 06 API 레퍼런스

Base: `http://localhost:8080` · 전 경로 CORS 개방 · 상세는 Swagger `/docs` .

| 메서드      | 경로                                      | 설명 · 요청 → 응답                                                                           |
|----------|-----------------------------------------|----------------------------------------------------------------------------------------|
| GET      | <code>/api/health</code>                | 게이트웨이 상태 → <code>{service,status,time}</code>                                          |
| POST     | <code>/api/scan</code>                  | <code>{target}</code> → <code>{target,job_id,kind,risk_score,grade,reasons[]}</code>   |
| GET      | <code>/api/graph</code>                 | 관계망(Neo4j, 폴백 시드) → <code>{nodes[],edges[]}</code>                                     |
| GET      | <code>/api/graph/expand</code>          | ?value= 노드 이웃 → <code>{nodes[],edges[]}</code>                                         |
| GET      | <code>/api/reports</code>               | 신고 목록(Postgres) → <code>[{id,target,kind,note,status,votes,ts}]</code>                 |
| POST     | <code>/api/report</code>                | <code>{target,kind,note}</code> 접수 → 엔진 위임                                             |
| POST     | <code>/api/reports/{id}/moderate</code> | <code>{status}</code> 확인/반려 → <code>{id,status}</code>                                 |
| GET      | <code>/api/search</code>                | ?q= 위협 검색(Meili) → <code>{query,hits[]}</code>                                         |
| GET      | <code>/api/stats</code>                 | 라이브 집계 → <code>{tracked_entities,graph_relations,scans_today,confirmed_threats}</code> |
| POST/GET | <code>/api/keys</code>                  | API 키 발급 / 목록 → <code>{key,owner,rate_limit,created_at}</code>                         |
| GET      | <code>/api/admin/analytics</code>       | 분석 집계 → <code>{by_grade,by_type,timeline[],totals}</code>                              |
| WS       | <code>/ws/feed</code>                   | 실시간 이벤트 스트림 → <code>{type,target,kind,grade,risk_score,note,ts}</code>                 |

## 07 디렉터리 구조

```

makertone/
├── docker-compose.yml          # 8개 서비스
├── Makefile                    # make up / seed / down
├── README.md · CLAUDE.md
├── apps/
│   ├── web/                   # Next.js 프론트
│   │   ├── app/
│   │   │   ├── page.tsx · layout.tsx · globals.css
│   │   │   ├── admin/page.tsx          # /admin 라우트
│   │   │   ├── components/
│   │   │   │   ├── CommandCenter · ScanConsole · GraphExplorer
│   │   │   │   ├── LiveFeed · ThreatMap · SearchBar
│   │   │   │   ├── StatsBar · ApiCallout
│   │   │   │   └── admin/AdminDashboard.tsx
│   │   │   └── data/ mockGraph.ts · mockThreats.ts
│   │   └── lib/ api.ts · feed.ts · search.ts · admin.ts
│   ├── gateway/              # Spring Boot
│   │   └── src/main/java/io/scamgraph/gateway/
│   │       ├── GatewayApplication · ScanController · GraphController
│   │       ├── ReportController · SearchController · SearchIndexer
│   │       ├── StatsController · ApiKeyController · AdminController
│   │       ├── RateLimitFilter · OpenApiConfig · FeedConfig/FeedHandler
│   │       └── Neo4jConfig · Neo4jGraphReader · ThreatCatalog
│   ├── engine/               # (Python, git 제외 - 담당자 코드)
│   └── extension/            # MV3 확장 (manifest·background·content·popup)
├── infra/
│   ├── neo4j/seed.cypher      # 관계망 시드
│   └── postgres/init.sql      # 스키마 + 시드 신고
└── docs/                      # 기획서 · 본 문서
  
```

## 08 실행 가이드

```

# 전체 스택 (8 서비스)
make up          # = docker compose up --build

# Neo4j 관계망 시드 (그래프 실데이터)
make seed

# 정지 / 로그
make down        # 컨테이너 제거 (데이터 볼륨 보존)
make logs
  
```

| 대상               | 주소                                                        |
|------------------|-----------------------------------------------------------|
| 메인 대시보드          | <code>http://localhost:3001</code>                        |
| 관리자 대시보드         | <code>http://localhost:3001/admin</code>                  |
| API 문서 (Swagger) | <code>http://localhost:8080/docs</code>                   |
| Neo4j 브라우저       | <code>http://localhost:7474</code> (neo4j / scamgraph123) |

### 브라우저 확장 로드

`chrome://extensions` → 개발자 모드 → “압축해제된 확장 프로그램 로드” → `apps/extension` 선택. (게이트웨이가 :8080에서 실행 중이어야 함)

**주의 — Postgres 초기화.** reports 등 테이블은 최초 볼륨 생성 시 init.sql 이 자동 실행돼 만들어진다. 다만 볼륨이 init.sql 실행 전에 생성된 상태로 재사용되면 테이블이 없을 수 있다. 이때는 수동 적용:

```
docker compose exec -T postgres psql -U scamgraph -d scamgraph -f /docker-entrypoint-initdb.d/init.sql
```

(없어도 API는 시드 폴백으로 동작하지만, 실패데이터를 쓰려면 필요.)

## 09 파트 분담

| 파트                                | 담당           | 범위                                         |
|-----------------------------------|--------------|--------------------------------------------|
| web + gateway + extension + infra | 우리 (저장소 포함)  | 프론트 전체 · Spring Boot 게이트웨이 · 브라우저 확장 · 인프라 |
| engine (Python)                   | 담당자 (저장소 제외) | 규칙 엔진 · 크롤러 · 워커 — 담당자 코드로 대체 예정           |

**계약(contract)** — 두 파트의 유일한 접점은 API 계약( docs/engine-spec.md §4 = docs/web-gateway-spec.md §4 ). 계약만 지키면 병렬 개발 가능. 엔진 담당자에게서 전체 코드를 받으면 apps/engine/ 에 덮어쓰면 되고, 게이트웨이·프론트는 그대로 연동된다.

현재 apps/engine/ 은 .gitignore 로 저장소에서 제외되어 있으며, 로컬 dev/test 코드가 디스크에 존재해 데모 실행이 가능하다. 계약 문서 docs/engine-spec.md 는 저장소에 유지된다.

## 10 데모 가이드 (킬샷)

**킬샷** — 심사위원이 아무 URL이나 입력 → 위험 게이지 + 근거 카드가 뜨고 → 관계망 그래프에 노드가 실제로 추가되며 → 실시간 피드에도 등장. “지금 실제로 인터넷을 분석하는” 장면.

### 시연 순서(권장)

- 메인 대시보드 오픈 — 공유 IP 허브로 얹힌 관계망 그래프를 먼저 보여준다(“두 캠페인이 한 서버를 공유”).
- 스캔 콘솔에 위험 도메인(예: kb-security-login.xyz ) 입력 → danger/근거 3종 즉시 표시.
- 그래프에 해당 노드가 추가되는 것 + 실시간 피드에 스캔 이벤트가 뜨는 것 지목.
- 검색 바에 shinhan 입력 → 자체 검색엔진 결과.
- deck.gl 위험 지도(공격 궤적) 스크롤.
- /admin 이동 — 차트 + 신고 모더레이션(확인/반려) 시연.
- /docs Swagger + API 키 발급 → “공개 API” 마무리.
- (선택) 브라우저 확장으로 위험 사이트 경고 배너 시연.

## 11 데모 세이프 원칙

모든 계층이 백엔드가 죽어도 화면이 유지되도록 설계되어 있다. 시연 중 무엇이 실패해도 데모가 무너지지 않는다.

- 규칙 엔진** — quick\_assess() 는 네트워크 없이 즉답. 크롤링 실패해도 위험도는 나온다.
- 게이트웨이** — 엔진/Neo4j/Postgres/Meili가 다운이면 시드·카탈로그·baseline으로 폴백. Hikari initialization-fail-timeout: -1 로 DB 없이도 기동.
- 프론트** — 그래프는 mockGraph, 피드/지도/stat/검색/관리자 모두 시드 폴백으로 절대 공백이 아니다. WebSocket 자동 재연결.
- 확장** — 게이트웨이 다운 시 페이지에 무해한 무동작.